

✓ Tutorial 1 - Environment Setup and Prompt Chaining

Date: 7 Jan, 2026

双击（或按回车键）即可修改

✓ Installing Packages

```
!pip install langchain-google-genai
```

```
Requirement already satisfied: langchain-google-genai in /usr/local/lib/python3.12/dist
Requirement already satisfied: filetype<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: google-genai<2.0.0,>=1.56.0 in /usr/local/lib/python3.12
Requirement already satisfied: langchain-core<2.0.0,>=1.2.5 in /usr/local/lib/python3.1
Requirement already satisfied: pydantic<3.0.0,>=2.0.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: anyio<5.0.0,>=4.8.0 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: google-auth<3.0.0,>=2.47.0 in /usr/local/lib/python3.12/
Requirement already satisfied: httpx<1.0.0,>=0.28.1 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: requests<3.0.0,>=2.28.1 in /usr/local/lib/python3.12/dis
Requirement already satisfied: tenacity<9.2.0,>=8.2.3 in /usr/local/lib/python3.12/dist
Requirement already satisfied: websockets<15.1.0,>=13.0.0 in /usr/local/lib/python3.12/
Requirement already satisfied: typing-extensions<5.0.0,>=4.11.0 in /usr/local/lib/pytho
Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: sniffio in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: jsonpatch<2.0.0,>=1.33.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: langsmith<1.0.0,>=0.3.45 in /usr/local/lib/python3.12/di
Requirement already satisfied: packaging<26.0.0,>=23.2.0 in /usr/local/lib/python3.12/d
Requirement already satisfied: pyyaml<7.0.0,>=5.3.0 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: uuid-utils<1.0,>=0.12.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/di
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: cryptography>=38.0.3 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: jsonpointer>=1.9 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: orjson>=3.9.14 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: requests-toolbelt>=1.0.0 in /usr/local/lib/python3.12/di
Requirement already satisfied: zstandard>=0.23.0 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/di
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: cffi>=1.12 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.12/dist-p
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages (fr
```

✓ Importing library

```
from google.colab import userdata
import os
```

In this tutorial, we will be playing with Gemini Models through [Gemini API](#).

✓ Gaining Access to Gemini API

- On registration, Gemini will give us 300USD of credits for free in 91 days. Go to docs.cloud.google.com then click Signing up for the [Google Cloud Free Trial](#). You will need a credit card to proceed.

Unfortunately, HK is not in the list of supported country :(

If you know how to change region or have credit card that belongs to those [accepted countries](#), you may proceed with this step, otherwise, you need to use Vertex API.

Welcome to your Free Trial

Build, test, and explore for free.
No hidden charges. No surprise bills.

 [Free usage amounts](#)  \$2,332 credits remaining  91 days remaining

 Upgrade for full access including customer support. Keep any remaining credits. [Upgrade](#)

✓ Vertex API

- Go to [Vertex AI Platform](#), click Try Vertex AI Free.
- Then go to [API KEY](#) and click create API Key

✓ Setup your API key

To run the following cell, your API key must be stored it in a Colab Secret named `GEMINI_API_KEY` or `VERTEX_API_KEY`.

Please be careful with your api key. You risk losing money if exposing your api key on the internet (see [here](#)). That's why we store API KEY in Colab Secret instead of pasting it into python variable.

```
GEMINI_API_KEY = userdata.get('GEMINI_API_KEY')
```

✓ Two ways of using your GEMINI KEY

using [Google Gen AI SDK](#)

```
from google import genai
client = genai.Client(api_key=GEMINI_API_KEY)
response = client.models.generate_content(
    model="gemini-2.5-flash-lite",
    contents="Explain how flash attention works in a two sentences.",
)
print(response.text)
```

Flash Attention optimizes attention computation by reorganizing the calculation to reduce memory usage.

Or using [langchain](#)

With google api, check your api key usage [here](#)

```
from langchain_google_genai import ChatGoogleGenerativeAI
llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash-lite",
    api_key=GEMINI_API_KEY,
    project="project-62a3e237-9dc9-4bdd-a6b",
    vertexai=False,
    temperature=0.2
)
```

with vertex api

```
# from langchain_google_genai import ChatGoogleGenerativeAI
# llm = ChatGoogleGenerativeAI(
#     model="gemini-2.5-flash",
#     api_key=userdata.get('VERTEX_API_KEY'),
#     vertexai=True,
#     temperature=0.2
# )
```

```
result = llm.invoke("hello world - What is CUHK stands for?")
print(result)
```

```
content='CUHK stands for **The Chinese University of Hong Kong**.' additional_kwargs={}
```

Since LangChain is designed for chaining prompts, this tutorial will focus on practicing prompt chaining with LangChain.

✓ Prompt Chaining 1 - Simple Example

```
import json
from typing import Dict, Any

from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser, JsonOutputParser
from langchain_core.runnables import RunnableLambda, RunnablePassthrough

# --- Prompt 1: Extract Information ---
prompt_extract = ChatPromptTemplate.from_template(
    "Extract the technical specifications from the following text:\n\n{text}
")

# --- Prompt 2: Transform to JSON ---
prompt_transform = ChatPromptTemplate.from_template(
    "Transform the following specifications into a JSON object with 'cpu'
")

# --- Build the Chain using LCEL ---
# The StrOutputParser() converts the LLM's message output to a simple string
extraction_chain = prompt_extract | llm | StrOutputParser()

# The full chain passes the output of the extraction chain into the 'spec
# variable for the transformation prompt.
full_chain = (
    {"specifications": extraction_chain}
    | prompt_transform
    | llm
    | StrOutputParser()
)

# --- Run the Chain ---
input_text = "The new laptop model features a 3.5 GHz octa-core processor,

# Execute the chain with the input text dictionary.
final_result = full_chain.invoke({"text_input": input_text})

print("\n--- Final JSON Output ---")
print(final_result)
```

```
--- Final JSON Output ---
```json
{
 "cpu": "3.5 GHz octa-core",
 "memory": "16GB",
 "storage": "1TB NVMe SSD"
}
```

```
}
...
```

Let's see whether this chain still works with a slightly longer input text.

```
slightly_longer_input_text = (
 "To reduce fraud scoring latency at the payment gateway, we're deplc
 "a 3.6 GHz octa-core CPU, 32GB RAM for feature stores and real-time
 "for event buffering and audit trails."
)

Execute the chain with the input text dictionary.
final_result = full_chain.invoke({"text_input": slightly_longer_input_text})

print("\n--- Final JSON Output ---")
print(final_result)
```

```
--- Final JSON Output ---
```json
{
  "cpu": "3.6 GHz octa-core",
  "memory": "32GB",
  "storage": "1TB NVMe SSD"
}
```
```

It still works ! Now let's make things more interesting by inputting a long document.

## ✓ Prompt Chaining 2 - Analyzing Xiaomi's Financial Report

### Assignment: Rebuild "Adjusted Profit" From Xiaomi's Interim Report (Prompt Chain)

Report PDF: [xiaomi's 2025 interim report](#)

You will build a **prompt chain** (multi-step agent) that reads a provided **plain-text extract** from a company report and produces:

1. a structured **calculation** that rebuilds the company's "Adjusted Profit" from ingredients
2. a short **plain-language explanation** of what was removed/added and why

## ✓ Load the financial report.

We provide the clean and short version of this report in `xiaomi.md`. We will study how to clean and chunk the full pdf in the `Retrieval Augmented Generation Lecture`.

```
import gdown
file_id = "1iQWfX7pKN_071txcqbJYWon8smBLQi0g"
download_url = f"https://drive.google.com/uc?id={file_id}"
gdown.download(download_url, "xiaomi.md", quiet=False)

Downloading...
From: https://drive.google.com/uc?id=1iQWfX7pKN_071txcqbJYWon8smBLQi0g
To: /content/xiaomi.md
100%|██████████████████| 10.2k/10.2k [00:00<00:00, 9.40MB/s]
'xiaomi.md'
```

```
with open("xiaomi.md", "r", encoding="utf-8") as f:
 markdown_text = f.read()
```

Let's see how to count tokens

```
response = client.models.count_tokens(
 model="gemini-2.5-flash",
 contents=input_text,
)

print(f"This prompt was worth {response.total_tokens} tokens.")

This prompt was worth 33 tokens.
```

```
response = client.models.count_tokens(
 model="gemini-2.5-flash",
 contents=markdown_text,
)

print(f"This prompt was worth {response.total_tokens} tokens.")

This prompt was worth 2734 tokens.
```

choosing the models: <https://ai.google.dev/gemini-api/docs/pricing>

## ✓ A simple prompt

```
Simple single prompt: ask only for the formula, nothing else
prompt_numeric_formulas = ChatPromptTemplate.from_template(
 """Using ONLY the markdown below:

1) Extract the reconciliation line items for 2025H1 and write the explicit
 that equals the reported Adjusted Net Profit for 2025H1.
2) a short plain-language explanation of what was removed/added and why
Rules for 1:
```

- Output EXACTLY one lines:  
Line 1 must start with "2025H1: "
- Each line must be a full equation with the reported total on the right  
2025H1: 1 + 1

Rules for 2:

- Output each term on one line, following by explanation for that terms

MARKDOWN:

```
{text_input}
```

```
"""
```

```
)
```

```
chain = prompt_numeric_formulas | llm | StrOutputParser()
```

```
result = chain.invoke({"text_input": markdown_text})
```

```
print(result)
```

```
2025H1: 22765909 + 2363735 - 4872245 + 72004 + 820911 + 355978 = 21506292
```

Share-based compensation (1): Added back 2,363,735 thousand RMB. This is a non-cash exp  
Net fair value changes on investments (2): Subtracted 4,872,245 thousand RMB. This repr  
Amortization of intangible assets from acquisitions (3): Added back 72,004 thousand RMB  
Changes of value of financial liabilities to investors (4): Added back 820,911 thousand  
Income tax effects (5): Added back 355,978 thousand RMB. This represents the tax impact

Using one prompt like this has limited capabilities:

- The LLM may be overwhelmed when instructions get complicated (format following, mathematical guarantee, ...)
- For complex tasks, if the LLM gives incorrect answers, you don't know which step to debug

That's why when presented with a complicated task, we decompose the instruction into chain of prompts.

## ✓ A chain using calculator and writing user report

Useful helper: RunnablePassthrough let you keep data between each element in the chain

```

1) Prompt: LOCATE (pull relevant parts only)

prompt_locate = ChatPromptTemplate.from_template(
 """You are a careful document assistant.
```

```
Given the markdown below, output ONLY a cleaned markdown excerpt that include
A) the definition of Adjusted Net Profit (Adjusted Profit),
B) the unit line (e.g., RMB '000),
C) the reconciliation table for 2025
```

~, the reconstruction table for 2025

D) the short notes that explain each adjustment item.

Rules:

- Do NOT add new facts.
- Keep it compact.
- Keep parentheses around negative numbers exactly as shown if present.
- Output markdown only (no commentary).

MARKDOWN:

```
{text_input}
"""
)
locate_chain = prompt_locate | llm | StrOutputParser()

2) Prompt: EXTRACT -> JSON (strict schema)

json_parser = JsonOutputParser()

prompt_to_json = ChatPromptTemplate.from_template(
 """Convert the markdown excerpt into a JSON object with this EXACT schema:

{{
 "unit": string,
 "periods": {{
 "2025H1": {{
 "profit_official": integer,
 "adjustments": {{
 "share_based_compensation": integer,
 "fair_value_changes_on_investments": integer,
 "amortization_from_acquisitions": integer,
 "value_change_financial_liabilities_to_investors": integer,
 "income_tax_effects": integer
 }}
 }}
 }}
 "adjusted_profit_reported": integer
}}
}},
"notes_plain_language": {{
 "share_based_compensation": string,
 "fair_value_changes_on_investments": string,
 "amortization_from_acquisitions": string,
 "value_change_financial_liabilities_to_investors": string,
 "income_tax_effects": string
}}
}}
}}
```

Rules:

- All money values must be SIGNED integers in the unit shown (e.g., RMB 'C
- If the markdown uses parentheses like (4,872,245), convert to -4872245.
- Remove commas from numbers.
- If any value is missing, put null (but try hard not to miss).

Return JSON ONLY.

MARKDOWN EXCERPT:

```
{excerpt}
```

```

"""
)
to_json_chain = prompt_to_json | llm | json_parser

3) Python CALCULATOR (no LLM math)

def compute_reconstruction(data: Dict[str, Any]) -> Dict[str, Any]:
 def recon_for(period_key: str) -> Dict[str, Any]:
 p = data["periods"][period_key]
 profit = p["profit_official"]
 adj = p["adjustments"]
 computed = profit + sum(adj.values())
 reported = p["adjusted_profit_reported"]

 abs_diff = None if (reported is None) else (computed - reported)

 return {
 "computed_adjusted_profit": computed,
 "reported_adjusted_profit": reported,
 "absolute_difference": abs_diff,
 }

 return {
 "unit": data.get("unit"),
 "results": {
 "2025H1": recon_for("2025H1"),
 }
 }

4) Prompt: EXPLAIN (grounded in computed numbers)

prompt_explain = ChatPromptTemplate.from_template(
 """Write a short explanation (max ~250 words) for a non-finance audience.

Rules:
- On first line, write the formula to compute the adjusted profit in the period.
- On second line, switch the numeric values for each period.
- Use ONLY the numbers provided below. Do not invent any.
- Explain what the "adjusted" score is trying to do vs the official score.
- For 2025H1: mention the top 1-2 biggest adjustments (by absolute value) and explain them.
- Explain each adjustment category in 1-2 simple sentences using the notes.

EXTRACTED DATA (JSON):
{data_json}

RECONSTRUCTION RESULTS (JSON):
{recon_json}

Return markdown only.
"""
)
explain_chain = prompt_explain | llm | StrOutputParser()

5) Build the full chain (LOCAL)

```

```

3) Build the full chain (LOCATE)

full_chain = (
 {"excerpt": locate_chain}
 | RunnablePassthrough.assign(extracted=to_json_chain)
 | RunnablePassthrough.assign(reconstruction=RunnableLambda(lambda x: compute_
 | RunnablePassthrough.assign(
 explanation=RunnableLambda(
 lambda x: explain_chain.invoke({
 "data_json": json.dumps(x["extracted"], ensure_ascii=False),
 "recon_json": json.dumps(x["reconstruction"], ensure_
 })
)
)
)

out = full_chain.invoke({"text_input": markdown_text})

print("\n--- Cleaned Excerpt (from LOCATE) ---\n")
print(out["excerpt"])

print("\n--- Extracted JSON (for grading) ---\n")
print(json.dumps(out["extracted"], indent=2, ensure_ascii=False))

print("\n--- Reconstruction Results (computed in Python) ---\n")
print(json.dumps(out["reconstruction"], indent=2, ensure_ascii=False))

print("\n--- Explanation (markdown) ---\n")
print(out["explanation"])

```

--- Cleaned Excerpt (from LOCATE) ---

Adjusted Net Profit is defined as \*profit for the period\*, as adjusted by adding back

1. Share-based compensation
2. Net fair value changes on investments
3. Amortization of intangible assets resulting from acquisitions
4. Changes of value of financial liabilities to investors
5. Income tax effects of non-IFRS adjustments

| (RMB in thousand, unless specified) | As reported | Share-based compensation (1) | Ne |
|-------------------------------------|-------------|------------------------------|----|
| Profit for the period               | 22,765,909  | 2,363,735                    |    |

1. Represents expenses related to share-based payments.
2. Primarily includes fair value changes on equity investments and preferred share in
3. Represents amortization of intangible assets resulting from acquisitions.
4. Represents the change in value of financial liabilities payable to investors.
5. Income tax effects of non-IFRS adjustments.

--- Extracted JSON (for grading) ---

```

{
 "unit": "RMB in thousand",
 "periods": {
 "2025H1": {
 "profit_official": 22765909,

```

```

 "adjustments": {
 "share_based_compensation": 2363735,
 "fair_value_changes_on_investments": -4872245,
 "amortization_from_acquisitions": 72004,
 "value_change_financial_liabilities_to_investors": 820911,
 "income_tax_effects": 355978
 },
 "adjusted_profit_reported": 21506292
 }
},
"notes_plain_language": {
 "share_based_compensation": "Represents expenses related to share-based payments.",
 "fair_value_changes_on_investments": "Primarily includes fair value changes on equity investments.",
 "amortization_from_acquisitions": "Represents amortization of intangible assets recognized in the period.",
 "value_change_financial_liabilities_to_investors": "Represents the change in value of financial liabilities to investors.",
 "income_tax_effects": "Income tax effects of non-IFRS adjustments."
}
}

--- Reconstruction Results (computed in Python) ---

{
 "unit": "RMB in thousand",
 "results": {
 "2025H1": {
 "computed_adjusted_profit": 21506292,
 "reported_adjusted_profit": 21506292,
 "absolute_difference": 0
 }
 }
}

```

## Try it yourself, now try to

- fix the prompt chain to report BOTH 2024 and 2025's adjusted profit.
- (python exercise) fix the compute\_reconstruction function so that `out["reconstruction"]` print out top 2 highest absolute values terms.

## ✓ Final notes

Agentic AI is evolving rapidly, making it impossible to cover every concept in a single tutorial. We encourage you to explore the resources below for a deeper understanding and practical guidance on building real-world applications:

- <https://github.com/google-gemini/cookbook>
- <https://docs.langchain.com/oss/python/learn>

We still have a long way to go. Real-world applications often require us to either

- design a set of super generic prompt chains that works for the entire application
- auto-matically generate the prompt chain instead of writing each one

We will learn how to route the input to the correct prompt chain in next week lecture / tutorial (Routing)

开始借助 AI 编写或生成代码。